PRODUCT BRIEF

CLI Shape for Agent Harnesses

Why agent-era CLIs need a measured interface contract, and where CLIARE fits.

CLIARE June 17, 2026 Audience: CLI maintainers, platform teams, agent harness builders

Executive Summary

Agent harnesses increasingly work in code mode: they inspect repositories, edit files, run tests, call local tools, and use the shell as the control plane. That is why terminal-first agents such as Claude Code, Codex CLI, and Gemini CLI matter. Their public documentation is explicit about reading code, editing files, running commands, and bringing models directly into the terminal.[1, 2, 3]

This creates a new interface problem. Many CLIs are being built or repositioned so agents can reach them. At the same time, many of those CLIs are private, new, version-specific, plugin-driven, or changed after a model was trained. The harness cannot rely on memorized syntax. It discovers the surface at runtime by reading help, trying commands, parsing errors, and adapting to local state. That discovery loop is often non-deterministic.

CLIARE's opportunity: become the neutral, evidence-backed shape layer for command-line interfaces: the CLI equivalent of an OpenAPI-style contract, but measured from the released executable and designed for local state, preconditions, side effects, and agent execution.

The Opportunity

The software industry already has standards for HTTP APIs. OpenAPI defines a language-agnostic interface description so humans and computers can understand an API without source access or traffic inspection.[4] CLIs do not have an equivalent operational contract.

That gap is becoming expensive because agents prefer interfaces they can execute inside the environments where software work happens: repositories, terminals, CI workers, containers, and developer machines. A CLI is often the shortest path to authenticated operational authority. It already has the user's credentials, project context, config files, local cache, and established deployment workflow.

The opportunity is not to replace CLIs. It is to make them measurable and queryable enough that maintainers can improve them and harnesses can use them without guessing.

The Problem

The current agent-facing CLI ecosystem has three failure modes.

1. Post-training surfaces

New CLIs, private CLIs, internal subcommands, plugin commands, and fast release cycles appear after a model's training run. A harness has to discover them live. Two runs can diverge because auth, working directory, installed plugins, environment variables, or probe order changed.

2. Human-readable is not machine-operable

A human can recover from missing examples, ambiguous preconditions, inconsistent exit codes, and output that changes shape. An agent usually pays for that with repeated discovery loops, invalid invocations, brittle parsing, or unnecessary escalation to the user.

3. Harnesses lack a standard query target

A harness needs to answer simple questions before executing a command: what command matches this intent, what arguments are required, what output mode is parseable, what preconditions are missing, what is safe to probe, and what sequence should come next. Today that shape is usually reconstructed from help text, docs, examples, and failures.

Why This Matters to Maintainers

Agent usage turns CLI design issues into product issues. A command with unclear preconditions can force agents to keep probing. A missing JSON mode can force fragile text parsing. A help command that mutates local state can make CI and harness execution unsafe. An invalid flag diagnostic that does not name the problem can waste multiple turns.

Maintainers need feedback that is direct enough to act on:

- This is the issue.
- This is the agent or harness outcome it causes.
- This is how to fix it.
- If it is intentional, record a disposition so it does not keep returning as unreviewed noise.

That is the same reporting direction CLIARE has moved toward: issue ledgers, persona packets, maintainer dispositions, and reports written around outcomes rather than internal scoring representations.

Why This Matters to Harnesses

Harness quality depends on the interface between the model and the environment. SWE-agent's research argues that agent-computer interface design affects software engineering agent performance, especially around repository navigation and command execution.[5] The same principle applies to third-party CLIs.

A harness should not have to rediscover a large CLI from scratch in every run. It needs a compact, deterministic, versioned surface it can query:

- **Intent routing:** map "check job status" to likely commands.
- **Invocation shape:** know the command path, required operands, flags, examples, and argument template.
- **Output contract:** prefer JSON or other parseable modes when available.
- **Readiness and risk:** see missing preconditions, cautions, safety notes, and evidence confidence before execution.
- **Version awareness:** cache shape only when the measured binary, profile, context, and artifact set still match.

How CLIARE Solves It

CLIARE treats a CLI as a black-box runtime system. It measures the released executable, not just source declarations or documentation. The measurement records evidence, infers command shape, scores readiness dimensions, and emits artifacts that can be reviewed by humans or consumed by machines.

```
cliare measure ./target/debug/mycli --out .cliare/mycli --profile standard
cliare issues list --out .cliare/mycli --format human
cliare surface query "check job status" --out .cliare/mycli --format json
cliare surface explain "jobs status" --out .cliare/mycli
```

The important shift is the surface resolver. The command index can be large. A harness should not have to read every row, infer which column matters, and reconstruct an invocation. It should ask a measured artifact directory for candidates and receive command paths, argument templates, output modes, preconditions, cautions, gaps, and evidence references.

That makes CLIARE useful in two loops: the maintainer improvement loop and the harness execution loop.

Maintainer Outcome

Before

- Agent failures arrive as vague bug reports.
- CI may catch regressions only after users complain.
- Docs, help text, parser behavior, and examples drift apart.
- Intentional behavior keeps reappearing as unresolved noise.

With CLIARE

- Issues are tied to measured evidence and concrete outcomes.
- CI can gate readiness regressions against a baseline.
- Preconditions, output modes, diagnostics, and safety behavior become visible.
- Reviewed dispositions keep accepted decisions explicit.

Harness Outcome

Before

- Probe help, try commands, parse errors, repeat.
- Depend on model memory of old or public syntax.
- Build one-off adapters for each CLI.
- Spend tokens and wall time on discovery instead of task execution.

With CLIARE

- Query measured shape by intent.
- Select parseable output modes before execution.
- Inspect preconditions and cautions up front.
- Cache and verify shape by binary fingerprint and measurement profile.

What the Standard Should Contain

A CLI shape standard should not stop at command names. A useful agent-facing contract needs:

- Command path, aliases, short description, examples, and intent terms.
- Arguments, flags, arity, defaults, conflicts, and required operands.
- Output contracts, including parseable modes and validation evidence.
- Preconditions: auth, project context, profile, fixture/input, plugin, dependency, or remote state.
- Safety signals: discovery side effects, destructive verbs, dry-run support, and caution text.
- Evidence references, confidence, binary fingerprint, measurement context, and schema version.

- Maintainer dispositions for accepted risk, intentional design, false positives, and fixture-gated findings.

OpenAPI reduced guesswork for HTTP APIs. CLIARE can do the same for command surfaces, with one difference: because CLI behavior depends on local runtime context, the standard should be evidence-backed and replayable, not just hand-authored.

Product Positioning

CLIARE should be positioned as the open standard and reference implementation for CLI shape measurement. It sits between maintainers and harnesses:

- **For maintainers:** a CI-ready quality gate and issue workflow for making a CLI easier for agents to operate.
- **For harnesses:** a deterministic shape query interface that reduces blind discovery and improves execution planning.
- **For ecosystems:** a common artifact format for comparing readiness, tracking drift, and publishing trustable scorecards.

This is complementary to existing tool protocols. MCP connects agents to external tools and data sources. OpenAPI describes HTTP APIs. CLIARE focuses on released command-line programs: the interface agents already reach when they operate in code environments.

Near-Term Product Work

- Stabilize the `surface` JSON schema so harnesses can depend on it.
- Add labeled intent corpora to evaluate ranking quality beyond string matching.
- Represent safe multi-command sequences, not just individual invocations.
- Publish a concise maintainer checklist for high-scoring agent-ready CLIs.
- Support scorecard publishing with provenance, versioning, and drift history.

The current string-ranked resolver is a useful first step. A BM25 or Tantivy-backed ranker may become valuable for Git-scale CLIs, but the standard should first lock down the shape schema, evidence model, and harness contract.

Selected Sources

1. Anthropic, "Claude Code Overview." The documentation describes Claude Code as an agentic coding tool that reads codebases, edits files, runs commands, and is available in the terminal. https://docs.anthropic.com/en/docs/claude-code/overview
2. OpenAI, `openai/codex` README. The repository describes Codex CLI as a coding agent from OpenAI that runs locally on a computer and installs as a terminal CLI. https://github.com/openai/codex

3. Google, "Gemini CLI: your open-source AI agent," June 25, 2025. Google describes Gemini CLI as an open-source AI agent that brings Gemini directly into developers' terminals. https://blog.google/technology/developers/introducing-gemini-cli-open-source-ai-agent/
4. OpenAPI Initiative, "OpenAPI Specification v3.2.0." The specification defines a standard, language-agnostic interface description for HTTP APIs. https://spec.openapis.org/oas/latest.html
5. John Yang et al., "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," arXiv:2405.15793, 2024. https://arxiv.org/abs/2405.15793
6. Shunyu Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv:2210.03629, 2022. https://arxiv.org/abs/2210.03629
7. The Open Group, "Utility Syntax Guidelines," POSIX Base Specifications, Section 12.2. https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap12.html
8. Supply-chain Levels for Software Artifacts, "SLSA Provenance," Version 1.2. https://slsa.dev/spec/v1.2/provenance